

Technology and Cost

Technology Brief for Ecomize — Multi-Tenant, Ultra-High Performance eCommerce SaaS

1. Core Goals

- **Sub-1 second vendor store load time** (core pages: home, category, product)
 - **Multi-tenancy:** Isolated vendor stores on shared infrastructure
 - **High concurrency:** Support thousands of simultaneous users
 - **Cost-efficient scaling & hosting**
 - **Robust redundancy, fault tolerance, and logging**
 - **Easy maintenance and smooth CI/CD pipeline**
-

2. Architecture Overview

2.1 Multi-Tenancy Model

Option	Description	Pros	Cons	Recommendation for Ecomize
Shared DB + Shared Schema	All vendors share same DB and tables; tenant_id in rows	Cost-efficient, simple setup	Harder to isolate noisy tenants, security risk	Suitable for MVP, but needs careful query optimization

Shared DB + Separate Schema	One DB, separate schemas per vendor	Good isolation, easier backups per tenant	Complex migrations, higher DB complexity	Good middle ground for scale and isolation
Separate DB per Tenant	Each vendor has own DB	Best isolation and security	Higher cost, complex orchestration	For top-tier customers or large vendors

For Ecomize: Start with **Shared DB + Separate Schema** to balance cost, isolation, and ease of scaling.

3. Frontend Optimization — Sub-1 Second Load

Framework & Techniques

- **Next.js with React + TypeScript** for SSR (server-side rendering) + static optimization (SSG, ISR).
- **Edge caching + CDN (AWS CloudFront or Cloudflare)** to serve static assets near users.
- **Incremental Static Regeneration (ISR)** for vendor store pages to reduce DB calls.
- **Critical CSS extraction + TailwindCSS** to minimize CSS payload.
- **Code splitting and lazy loading** for non-critical components.
- **Preconnect and DNS Prefetch** to speed up resource fetches.
- **HTTP/2 or HTTP/3** enabled for multiplexing requests.
- **Image optimization (Next/Image or custom loader)** for fast image delivery.
- **Client-side hydration** only on interactive parts for faster TTI (Time to Interactive).

4. Backend Architecture

Microservices

- **API Gateway** (e.g., AWS API Gateway, Kong) routes requests to microservices.
- Separate microservices for:
 - **Product catalog service** (fast reads, elastic search optimized)
 - **Order & cart service**
 - **User & authentication service**
 - **Payment gateway integration**
 - **Vendor management service**
 - **Notification & email service**
 - **Fraud detection service**

Databases

- **Relational DB** (MySQL or Aurora Serverless) with **Prisma ORM** for transactional data.
 - **Elasticsearch** for catalog search and filtering for ultra-fast queries.
 - **Redis/Memcached** for caching hot data like sessions, popular product data.
 - **MongoDB** for logs, audit trails, analytics (NoSQL flexibility).
-

5. Scaling & Hosting

Infrastructure

- Host on **AWS** for mature services and global footprint.
- Use **Kubernetes (EKS)** or **serverless containers (AWS Fargate)** for microservices orchestration.

- **Auto-scaling groups** for stateless services based on CPU/requests.
- Use **Aurora Serverless** for DB scaling with auto pause/resume to save cost.
- Leverage **CloudFront CDN** and caching strategies to offload traffic from backend.

Load Balancing & Traffic Management

- Use **AWS ALB (Application Load Balancer)** or **NGINX Ingress Controller** in K8s cluster.
 - Geo-routing with CloudFront + Route53 latency-based DNS.
 - Graceful shutdowns with rolling deployments for zero downtime.
-

6. Caching Strategy

- **Edge cache** with TTL for store pages (using CDN).
 - **Application-level caching** (Redis) for product details, cart sessions.
 - **Database query caching** for common queries.
 - Use **Cache Invalidation** on product/order updates via event-driven architecture (Kafka/SQS).
-

7. Logging & Monitoring

- Centralized log aggregation using **ELK Stack (Elasticsearch, Logstash, Kibana)** or **AWS CloudWatch Logs**.
- Distributed tracing with **OpenTelemetry** for performance bottleneck detection.
- Alerting via **Prometheus + Grafana** or AWS CloudWatch Alarms.

- Log retention policies for compliance and debugging.
-

8. Security & Fault Tolerance

- **Isolate tenant data** via schema separation and RBAC at app level.
 - **Rate limiting & throttling** to prevent abuse.
 - **WAF (Web Application Firewall)** like AWS WAF or Cloudflare to block attacks.
 - Secure APIs with **OAuth 2.0 / JWT tokens**.
 - **Multi-AZ DB clusters** for high availability.
 - Daily backups + point-in-time recovery.
 - Graceful degradation: serve stale cached data during backend failure.
-

9. Cost Optimization

- Use **serverless compute (AWS Lambda/Fargate)** for low-to-medium traffic microservices to reduce idle resource cost.
 - Use **auto-scaling** with set max limits to prevent runaway cost.
 - Optimize DB cost with **Aurora Serverless** or use **read replicas** for heavy reads.
 - Use **CloudFront with caching** to minimize origin hits.
 - **Spot Instances** or savings plans for predictable workloads.
 - Implement **monitoring with budgets** and automated alerts to keep spending in check.
-

10. Continuous Integration & Deployment (CI/CD)

- Use **GitHub Actions** / **AWS CodePipeline** for automated builds, tests, and deployments.
- Canary and blue/green deployments to avoid downtime.
- Automated DB migration scripts with rollback support.
- Unit + integration + load tests as part of pipeline.

Summary Technology Stack for Ecomize

Layer	Technology
Frontend	Next.js (React), TypeScript, Tailwind CSS
Backend	Node.js (Express/Koa), TypeScript, Microservices
DB	Amazon Aurora MySQL (Serverless), Elasticsearch, Redis
Infrastructure	AWS EKS (Kubernetes), Fargate, CloudFront CDN, Route53
Messaging/Queue	AWS SQS / Kafka
Logging/Monitoring	ELK Stack or CloudWatch + Prometheus + Grafana
Security	AWS WAF, OAuth 2.0 / JWT, RBAC
CI/CD	GitHub Actions / AWS CodePipeline

How this meets your goals:

Goal	Implementation Highlight
Sub-1 second load	Next.js ISR + edge caching + CDN + Redis cache + Elasticsearch

Multi-tenancy with isolation	Shared DB with separate schemas + RBAC
High concurrency	Microservices + Kubernetes autoscaling + caching
Cost efficiency	Serverless DB + Fargate + spot instances + caching
Reliability & redundancy	Multi-AZ DB + ALB + multi-region CDN + centralized logging
Scalability & maintenance	API gateway + microservices + GitHub Actions CI/CD

Here's a clear strategy to **manage traffic quotas per package** and **optimize infrastructure costs**:

1. How to Manage Visitor Traffic per Customer Store

A. Define Visitor Quotas by Package

- **Normal:** 50k visitors/month
- **Medium:** 100k visitors/month
- **Advance:** 200k visitors/month
- **Enterprise:** Custom (high volume)

B. Track Usage

- **Use a lightweight visitor counting system** per vendor store:
 - Track unique visitors per store via a fast analytics service or custom logging (e.g., Redis counters keyed by vendor + date).

- Increment counts on every unique session or IP per day.
- **Integrate visitor limits into backend API** that serves store pages:
 - If limit is near exceeded, notify vendor or throttle (show “upgrade plan” message or static cached pages).

C. Enforce Limits with Caching & Throttling

- **Cache aggressively on CDN level** so visitors don't hit your origin repeatedly:
 - For normal visitor traffic, 80–90% requests served from CDN cache — minimal server load.
- **Throttle backend dynamic requests** for vendors nearing quota.
- **Pre-generate static pages** (ISR in Next.js) to serve via CDN wherever possible.

D. Usage Reporting & Alerts

- Provide real-time dashboard for vendors to see visitor usage.
- Automated emails or in-app notifications when usage reaches thresholds.
- Offer smooth upgrade path to higher-tier packages before hard limits.

2. How to Handle High Traffic Without Burning Huge Server Cost

A. CDN-First Architecture

- Use a **global CDN (CloudFront, Cloudflare, etc.)** as first line:
 - Most visitor requests are served **directly from edge cache**.
 - Reduces load on origin servers by ~80–90%.

B. Incremental Static Regeneration (ISR)

- Use Next.js ISR to **pre-build and cache pages statically** but update them incrementally.
- For example:
 - Product pages, category pages regenerated every few minutes or when product updates happen.
 - Reduces DB queries and server processing.

C. Multi-Tier Caching

- **Edge CDN Cache** — for public pages.
- **Application Cache (Redis)** — cache frequently accessed data (product info, pricing).
- **Database Query Cache** — for expensive queries.
- This layered caching reduces DB and backend hits.

D. Serverless & Auto-Scaling Infrastructure

- Use **serverless functions (AWS Lambda/Fargate)** for backend APIs — pay per use, no idle cost.
- **Kubernetes auto-scaling** for microservices, scaling out during peaks, scaling down during idle.
- Use **Aurora Serverless** or similar for databases to scale with demand and pause when idle.

E. Visitor Quota-Based Throttling

- **Prevent abusive spikes** with rate limiting per vendor.
- For vendors near quota, restrict heavy features or API calls.
- Serve stale cached content with warning rather than dynamic content when near quota.

F. Prioritize High-Value Customers

- For Enterprise plans, **dedicated resources or DB instances** can be allocated.
 - Normal & medium packages share infrastructure with limits.
 - Enterprise plans get SLA-backed, reserved compute capacity.
-

3. Cost Optimization Strategies

Strategy	Benefit
Heavy CDN caching	Offload 80-90% traffic, minimize origin hits
Serverless compute	Pay only for requests processed
Auto-scaling Kubernetes	Scale horizontally only when needed
Incremental static builds	Pre-render pages, reduce server CPU/DB usage
Tiered visitor throttling	Avoid overspending on low-paying customers
Usage-based pricing	Align customer cost with usage, avoid free-riding

4. Example Traffic Management Flow

1. **Visitor requests vendor store URL.**
 2. CDN checks cache; if cached, serve immediately (zero server cost).
 3. If cache miss, backend checks visitor count for that vendor.
 4. If visitor count < quota, serve dynamic content, update counters, cache page.
 5. If visitor count > quota, serve cached stale content or “Upgrade Plan” message.
 6. Vendor sees dashboard alerts and can upgrade to increase quota.
-

5. Summary: How to Guarantee Visitor Limits Without High Costs

Aspect	Approach
Visitor tracking	Lightweight counting in Redis or analytics
Traffic serving	CDN + ISR + caching layers
Backend scaling	Serverless + auto-scaling Kubernetes microservices
Quota enforcement	Throttling + cached stale pages
Cost control	Usage-based pricing + multi-tier plans

If you want, I can help you design the detailed caching strategy, visitor quota enforcement logic, or cost simulation models next!

Example of Cost

Great! Starting with a **\$50/month server budget** (including compute, DB, CDN, logging, etc.) for an eCommerce SaaS platform, here's a rough estimate of how many vendors and visitors you can serve, given you apply strong caching and cost optimization techniques:

Assumptions & Constraints

- **\$50/month budget** covers:
 - Small AWS EC2 or equivalent VPS (~\$15–20)
 - Managed database (AWS Aurora Serverless free tier or small RDS instance ~\$15–20)
 - CDN (Cloudflare free plan or minimal paid tier) ~\$5
 - Redis cache (small Redis managed instance) ~\$5

- Logging/monitoring minimal cost (basic CloudWatch or open source self-hosted)
- Bandwidth & data transfer included or low volume initially
- **Platform optimization:**
 - Heavy CDN caching (~80%+ cache hit ratio)
 - Incremental Static Regeneration (ISR) or static page caching
 - Serverless or minimal backend usage for dynamic API calls
- **Visitor distribution:** Visitors spread across vendors evenly or skewed moderately.

Estimated Capacity

Metric	Estimate (Conservative)
Average Vendor Store Page Size (HTML + assets)	~300 KB (compressed, cached via CDN)
Average Visitor Requests per Session	~5 page views
Cache Hit Rate on CDN	80%+ (4 out of 5 page requests served from CDN)
Dynamic Requests per Visitor	1 page request hits your backend per session (due to caching)
Server CPU/RAM	1 vCPU / 2GB RAM instance (AWS t4g.micro or similar)
Database Load	10–20 queries per dynamic request (optimized with cache and indexing)

Visitor Handling Breakdown

Budget Allocation	Units	Notes
Server + App Instance	\$20/month	Handles ~200–300 dynamic requests per minute (4-5 req/sec)

Database (small RDS/Aurora)	\$15/month	Can handle 5,000–10,000 queries per minute if optimized
CDN (Cloudflare Free/Basic)	\$5/month	Offloads 80%+ traffic
Redis Cache (small instance)	\$5/month	Improves backend DB/cache hits
Logging/Monitoring	\$5/month	Basic log storage and metrics

Visitors per Month

- **Backend dynamic requests per month** (assuming 20% of total visits hit backend):
\$20 server + \$15 DB capacity ~ 5000 dynamic requests/min * 60 * 24 * 30 = ~216 million dynamic requests per month (theoretical max).
 - **Realistic dynamic requests:** Given CPU/RAM and DB limits, handle **around 100k dynamic requests/day** comfortably (with caching).
 - **Visitor sessions per day** (assuming 1 dynamic request per session): ~100k visitors/day backend load capacity.
 - **Visitor sessions per month:** 100k * 30 = **3 million visitors per month** possible backend-wise, if caching & CDN work well.
-

Vendor Count

- Assume average vendor gets 1,000 visitors/month initially (small local SMEs).
 - With 3 million visitors/month capacity, **~3,000 vendors** can be served within this budget.
 - Vendor store static assets served from CDN — no extra backend cost.
 - Database designed to support multi-tenant queries with caching reduces cost.
-

Important Notes

- These are **optimistic upper bounds** assuming heavy caching and even traffic distribution.
- Real-world traffic spikes, complex queries, product updates, and poorly cached content will reduce capacity.
- Logging volume should be controlled to avoid cost spikes.
- Bandwidth costs vary regionally; use CDN providers with generous free plans.

Scaling Plan

Visitors / Vendors Exceeding Budget	Next Step
> 50k visitors / 3,000 vendors	Upgrade instance size, increase Redis & DB tier
> 100k visitors / 5,000 vendors	Add auto-scaling Kubernetes cluster + managed DB
> 500k visitors / 10,000 vendors	Implement sharded DB, multi-region CDN
> 1M+ visitors	Consider dedicated DB per large vendor, edge compute

Summary

Budget	Vendors Supported	Visitors Supported	Notes
\$50/month	~3,000	~3 million/month	With heavy CDN caching & optimizations
\$100/month	~5,000	~5 million/month	Upgrade DB + Redis + compute
\$500/month +	10,000+	10M+	Scale Kubernetes + DB sharding

1. Custom Domain Handling (Shopify-like)

When a vendor connects their custom domain (`store.vendor.com` or `vendorstore.com`), your platform should serve their store with their theme/data on that domain.

✓ How It Works

1. DNS CNAME or A Record Setup by Vendor:

- Vendor sets a CNAME (e.g., `vendorstore.com` → `platform.deshi.store`) or points their A record to your platform IP.

2. Wildcard SSL Support:

Use a wildcard SSL cert via **Let's Encrypt** + automated provisioning (e.g., via certbot or Greenlock) OR use a wildcard reverse proxy domain like:

CopyEdit

`*.customer.yourplatform.com`

○

3. Host Header Based Routing:

- When a request comes in, extract the `Host` from headers.
- Look up the vendor by custom domain in your DB.
- Fetch/store their settings/theme/product data.

4. Serve Their Store:

- Use their theme and data to render via Next.js SSR/ISR or serve static pre-built pages via CDN (if optimized).

✓ 2. Super Admin Control Over Vendor > Server Mapping

When you scale, you might want to assign specific vendors to specific instances or clusters (e.g., heavy traffic vendors → stronger server).

✓ How to Handle That

1. Centralized Vendor Registry DB (Platform Core DB):

A master DB contains:

```
json
CopyEdit
{
  "vendor_id": "xyz",
  "custom_domain": "vendorstore.com",
  "assigned_instance": "cluster-a",
  "plan": "enterprise"
}
```

○

2. Reverse Proxy/Router (NGINX, HAProxy, or Custom Node Router):

- On each incoming request, check the vendor → instance map.
- Route the request to the correct backend/server instance or microservice.

3. Next.js Multi-Zone or API Gateway Layer:

- Use a central gateway (like **API Gateway**, **Express reverse proxy**, or **NGINX**) to direct traffic.
- Each instance/microservice hosts vendors of specific plans.

✓ 3. Tech Stack Summary (with Multi-tenancy & Domain Routing)

Purpose	Tech / Service
Backend	Node.js + TypeScript (Express, etc)
Frontend Rendering	Next.js (ISR, SSR, SSG mix)
Database	MySQL (Core Data), MongoDB (NoSQL)
ORM	Prisma

File Storage	AWS S3 / Cloudflare R2
CDN / Edge Cache	Cloudflare CDN / AWS CloudFront
SSL Certs	Let's Encrypt (via certbot or auto TLS)
Server Router / Proxy	NGINX or Node Proxy
Container / Scale	Docker + ECS / Fargate / EC2 / Lightsail
Logging	CloudWatch / ELK Stack / Logtail
Load Balancer	AWS ALB / NGINX / HAProxy
Domain Routing	Host Header logic (or nginx virtual host)
Analytics/Monitoring	Grafana, Prometheus, Vercel Insights

✓ 4. Cost-Smart Scaling Suggestions

Since you want to start with a ~\$50/month budget, here's how to begin and then scale:

Start Small (Initial Setup for \$50)

Resource	Plan/Service	Approx. Cost
Backend API Server	Hetzner 2vCPU 4GB + SSD VPS	\$10/month
CDN & SSL	Cloudflare Free + SSL/TLS	Free
Database (MySQL)	Railway.io, NeonDB or PlanetScale	Free–\$10
Object Storage	Cloudflare R2 or Backblaze B2	Pay-as-you-go
Static Hosting (Store)	Vercel / Netlify (Starter Tier)	Free
Logging & Analytics	Logtail / BetterStack / Grafana	Free–\$10

You can easily support ~100–200 active vendors with ~5–10k monthly visitors each (depending on store size, assets) under this budget with optimization + caching.

✓ 5. Summary

-  **Custom domain support** is done via DNS pointing, host header-based routing, and TLS automation.
-  **Super admin mapping** uses centralized vendor registry and routing layer.
-  **Scaling** is cost-effective with Docker, CDN, and static rendering.
-  **You can assign vendors to clusters/servers** via DB mapping or API gateway logic.

Stats



Ecomize Cost vs Scale Projection & Infrastructure Sizing Plan

1. Spreadsheet Structure Overview

Column	Description
Monthly Budget (\$)	Total budget allocated
Vendors Supported	Estimated number of vendors supported
Visitors per Month	Estimated visitor sessions handled per month
Backend Dynamic Requests	Estimated dynamic API calls handled per month
Compute Resources	Type & count of servers/containers
DB Resources	DB instance type & size
Cache Resources	Redis/memcached size & type
CDN Cost	Estimated CDN bandwidth cost
Logging & Monitoring	Estimated cost
Other Costs	Miscellaneous (e.g., backups, licensing)
Notes / Remarks	Special notes or scaling considerations

2. Sample Data Table

Tier	Budget (\$)	Vendors Supported	Visitors / Month	Backend Requests / Month	Compute	DB	Cache	CDN Cost	Logging	Other	Notes
------	-------------	-------------------	------------------	--------------------------	---------	----	-------	----------	---------	-------	-------

Starter	50	3,000	3,000,000	600,000 (20%)	1 x t4g.micro (1 vCPU, 2GB)	AWS Aurora Serverless db.t3.small	500 MB Redis	\$5	\$5	\$5	Heavy CDN caching, simple app, shared DB schema
Small Business	100	5,000	5,000,000	1,000,000 (20%)	1 x t4g.small (2 vCPU, 4GB)	AWS Aurora Serverless db.t3.medium	1GB Redis	\$10	\$10	\$10	Add auto-scaling, better monitoring
Growing	250	10,000	10,000,000	2,000,000 (20%)	2 x t4g.small (4 vCPU, 8GB)	AWS Aurora Serverless db.t3.large	2GB Redis	\$15	\$15	\$15	Kubernetes cluster + read replicas
Mid Enterprise	500	20,000	20,000,000	4,000,000 (20%)	4 x t4g.medium (8 vCPU, 16GB)	AWS Aurora db.r5.large	4GB Redis	\$20	\$20	\$20	Sharded DB, auto-scaling Kubernetes
Enterprise	1000+	50,000+	50,000,000+	10,000,000+ (20%)	K8s cluster 10+ nodes	Multi-region Aurora + shards	8GB+ Redis	\$40+	\$40+	\$40+	Dedicated resources + multi-region redundancy

3. Explanation & Justification

Backend Requests

- We estimate ~20% of visitors make a **dynamic backend request** due to caching.
- Example: 3M visitors × 20% = 600k dynamic requests/month.

Compute Resources

- Use burstable ARM instances (t4g series) on AWS to reduce cost.
- Scale from single micro instance to multiple instances or containers managed in Kubernetes.
- Use auto-scaling to expand capacity during peak times.

Database

- Start with Aurora Serverless or small RDS instance.
- Upgrade DB size and add read replicas or shard DB as you grow.
- Use schema-per-tenant or separate DB for large vendors.

Cache

- Use Redis for session caching, rate limiting, visitor counters, and DB query caching.
- Scale Redis memory with traffic growth.

CDN

- Most traffic served from CDN with high cache hit ratio (80%+).
- Costs grow with bandwidth and edge location usage.

Logging & Monitoring

- Start with basic CloudWatch / open-source ELK stack.

- Add Prometheus/Grafana with scale.
 - Keep logs at manageable retention to save cost.
-

4. How to Use This Projection

- Begin at Starter tier (\$50), monitor actual visitor traffic and resource usage.
- Track cache hit ratio and optimize to keep backend requests low.
- Increase budget as vendor count & visitors grow.
- Scale compute, DB, and cache horizontally as needed.
- Adjust CDN plan and logging retention policies to balance cost and performance.

Core Features

Super Admin Panel Features

(Platform Owner / Root Admin)

Platform & Vendor Management

- Approve / Reject / Suspend Vendors
- Create, edit, or delete vendor accounts
- Assign vendor to specific server/instance
- Set store limits (product, storage, traffic, bandwidth)
- Vendor impersonation / view-as-vendor

Subscription & Billing

- Create/manage pricing packages (Free, Starter, Pro, Advanced, Enterprise)
- Package-based limitations (products, traffic, bandwidth, staff accounts)
- Set pricing in multiple currencies
- Automatic billing and renewal management
- Usage tracking (disk space, bandwidth, visits)
- Alert vendors when nearing limits
- Invoicing and tax support (VAT, GST)
- Global Payment gateway for Platform and Merchant as Well (means merchant can use platform payment gateway instead of their own gateway as well). And Superadmin will withdraw payment manually with records.
-

Infrastructure Management

- Assign vendors to specific instances or clusters

- Server load monitoring
- Trigger scale-up/down or auto-scaling
- CDN configuration and asset purging
- Redis cache purging or warming
- Failover strategy and backup management
- Load balancer distribution and rules

Domain & DNS Management

- Custom domain support with DNS validation (like Shopify)
- SSL provisioning via Let's Encrypt or custom cert
- Domain mapping per vendor

Admin Store & Data Access

- View all store data (products, orders, users)
- Moderate content (reports/flags, banned items)
- Access to logs and activity tracking
- Data anonymization options for GDPR

Platform Analytics

- Total vendor count
- Platform-wide order volume
- Monthly Recurring Revenue (MRR) tracking
- Error rate, latency, uptime, and incident logging

User & Staff Management

- Admin staff roles with permissions
- Audit log of staff activities
- Ban/blacklist abusive users or fraudulent vendors

Security Features

- 2FA for super admin and vendor accounts
- CAPTCHA and brute-force protection
- Anti-fraud triggers & flagging system
- Rate-limiting per store/API
- Automated backups and restore options
- Web Application Firewall (WAF)

Notifications & Communication

- Global announcements
- Email templates for vendor alerts
- SMS gateway integration (e.g., for alerts)
- Integration with Helpdesk or Live Chat for support

Theme Management

- Manage theme for vendors
- Active deactivate themes
- Uploads theme and rollout for merchant

Vendor Panel Features

(Merchants running stores)



Store Management

- Enable/disable store
- Customize store name, URL, logo, and branding
- Manage custom domain (DNS setup, SSL status)



Storefront Builder & Pages

- Drag-and-drop store builder (section-based or theme-based)
- Theme preview and publish
- Live storefront editor
- Add/remove sections (product slider, hero banner, testimonial, etc.)
- Can Create/edit/delete pages with plain text or simple page builder (section based only)



Product Management

- Add/edit/delete products
- Bulk upload via CSV
- Product variants (size, color, etc.)
- Digital product support
- Product scheduling (launch later)
- Inventory tracking and low stock alerts



Order & Payment Management

- Order listing and Order details with all history and activity
- View orders by status (pending, processing, shipped)

- Manual and auto fulfillment options
- Generate invoices
- Integration with payment gateways (SSLCommerz, Stripe, bKash, Nagad, etc.)
- Refund and return handling
- COD and advanced shipping setup

Customer

- Customer List
- Customer CRUD operation

Customer

- Customer List
- Customer CRUD operation

Shipping & Delivery

- Shipping zones and rates
- Third-party logistics integration (e.g., Pathao, Paperfly)
- Track shipments
- Pickup location support

Inventory & Stock

- Centralized inventory tracking
- Stock history logs
- Product batching support
- Supplier wise CRUD Operations

Marketing & Sales

- Discounts, coupons, BOGO offers
- Flash sale setup
- Email marketing integration (e.g., Mailchimp)
- Popup campaigns and exit intent
- Social media links and auto-sharing
- Abandoned cart recovery



Analytics & Reports

- Sales summary
- Top-selling products
- Visitor tracking (integrate GA4)
- Heatmaps and session replays (via Hotjar/Clarity)
- Conversion tracking
- Storefront wise User Page journey like where they stops till complete order or before they leave. Why leave.
- Abandon cart and recovery.



SEO & Meta

- SEO editor per page/product
- Meta title, description, canonical URL
- Sitemap.xml auto-generation
- OpenGraph support for social sharing



Tax, Invoice & Legal

- Add tax rules by region

- VAT/GST setup
- Downloadable invoice PDF
- Add terms & conditions, privacy policy pages

Customer Engagement

- Live chat widget
- Contact form
- Testimonials and reviews
- Loyalty point system

Staff & Roles

- Add multiple staff members
- Set roles and permissions (orders only, product only, full access)

Integrations

- Advance Server side tracking Integration (will be added on different server via readymade scripts)
- Facebook Pixel
- GA4 & Google Tag Manager
- Meta Catalog, TikTok, Snapchat Pixels
- Webhook configuration
- API key management for custom integrations

Security

- 2FA for vendors

- CAPTCHA for login, forms
- Login activity and logs



Backup & Export

- Export products/orders/customers
- Data portability (to CSV, JSON, etc.)



AI Features (Optional / Roadmap)

- AI product description generator
- AI image enhancement
- AI-based upsell/cross-sell recommendation
- Smart inventory forecast
- AI chatbot for customer service



Required: Anti-Fake Order Protection (NEW)

- OTP verification for COD orders
- Spam detection via IP/Device fingerprinting
- Max order attempts per IP/day and hour and minutes.
- Flag suspicious high-value orders
- Blacklist by phone/email
- CAPTCHA after multiple failed attempts
- Use ML to identify unusual patterns (enterprise plan)
- BD Fraud Checker API for checking orders

Features



MULTI-TENANCY IMPLEMENTATION STRATEGY



Recommended Model: Database-per-Tenant with Shared Application Code

- **Database Layer:** Each vendor gets a separate database/schema. Ensures data isolation, scalability, and easier backups/restores.
 - **Application Layer:** Shared codebase (Node.js backend) with routing middleware to determine the tenant based on subdomain or request headers.
 - **Frontend (Storefront):** Static-rendered frontend (React/Next.js with ISR or Astro) delivered via CDN for near-zero load time.
 - **CDN:** Storefronts + assets (images, styles, product images) hosted via **Cloudflare CDN** or **Bunny CDN**.
-



INFRASTRUCTURE STACK (Cost-Efficient but Scalable)

Component	Technology	Notes
Backend	Node.js (Express or NestJS)	API Gateway w/ rate limiting
Frontend	Next.js / Astro (Static Export)	ISR/SSG, fast & SEO friendly
Database	PostgreSQL / PlanetScale (MySQL)	Per-tenant DB
Cache	Redis / Upstash	Session, Product Cache
Storage	Cloudflare R2 / Bunny.net	Cheaper than S3
Auth	Clerk/Auth0/FusionAuth or custom	OAuth + Email/OTP login
CDN	Cloudflare/Bunny CDN	Asset + HTML edge caching
Queue	BullMQ / RabbitMQ	For sending emails, stock sync, webhooks

Monitoring	Sentry, Grafana, Prometheus	Alerting + Debugging
Search Engine	Meilisearch / Algolia	Fast product search
Email	Resend / Postmark / Mailgun	Transactional + marketing
Load Balancing	NGINX / Cloudflare + Auto-scaling	Routes traffic per load

ADVANCED SaaS FEATURES

FOR SUPER ADMIN PANEL

Tenant/Vendor Management

- View/Add/Edit/Delete vendors
- Approve or suspend vendor access
- Assign custom plans & quotas (e.g., product limits, traffic caps)

Infrastructure Monitoring

- Real-time server load, tenant DB size
- Per-tenant traffic stats & usage alerts
- Error tracking & performance logs (Sentry/Grafana)

Billing & Invoicing

- Stripe integration (auto billing)
- Plan customization
- Manual adjustments, invoice exports

Analytics Dashboard

- Traffic, order volume, uptime by vendor

- Customer conversion rate stats

Support System

- Admin messaging & support tickets
- Auto/escalated priority system

Global Content Control

- Control homepage banners, blogs, pricing page, FAQ
- Announcements (shown on vendor dashboards)

Security & Access Control

- 2FA for staff
- Role-based access for internal team
- IP restrictions for staff access

Feature Toggle System

- Enable/disable beta features per vendor
- A/B testing modules per segment

Custom Domain Routing

- Map custom domains via Cloudflare API
- SSL Auto-provisioning (Let's Encrypt/Cloudflare SSL)



FOR VENDORS PANEL

Storefront Builder

- Drag & drop layout builder
- Custom CSS/JS (per plan)
- Template switching
- Theme customization (fonts, sections, colors)

Product Management

- Add/Edit/Delete Products (single or bulk)
- Variants, SKUs, barcodes, tags
- Stock tracking, low-stock alerts

Order Management

- View/Pending/Cancelled/Completed orders
- Export CSV
- Shipping status update
- Invoice generation

Customer Management

- View customers
- Tag/segment customers (e.g., repeat buyers)
- Export customer data

Analytics

- Live visitor tracking
- Conversion funnel, best-selling items
- Abandoned cart insights

- Product view heatmap (optional advanced add-on)

Marketing Tools

- Coupons, flash sales
- Abandoned cart recovery emails
- SEO optimization tools (custom meta, sitemap.xml)
- Blog/news module

Payment & Shipping

- Payment integration setup (SSLCOMMERZ, Stripe, bKash, Nagad)
- Shipping zone configuration
- Delivery cost per location

Mobile App Builder (Premium)

- Auto-generate PWA version
- Optional native app generation via wrapper

Apps & Extensions (Modular System)

- Enable add-ons like:
 - Affiliate system
 - Loyalty points
 - WhatsApp chat
 - In-store pickup
 - Multi-language support

Security

- 2FA for vendor login
- Staff roles and permissions (per store)
- Backup & Restore (premium plan)

Custom Domain Setup

- Guided DNS instructions
- Auto SSL provisioning (Cloudflare API)

Traffic Quota Monitoring

- Real-time visitor stats
- Alerts if close to quota limit (e.g., 50K/month)
- Purchase additional quota



ADDITIONAL INNOVATIVE FEATURES TO STAND OUT

1. **AI Product Description Generator** (based on product title + image)
2. **Mobile App Push Notification Manager**
3. **Built-in Product Image Optimizer**
4. **Live Chat Plugin** (integrated or 3rd-party like Crisp)
5. **Social Selling** (Product sharing via WhatsApp, Facebook shops)
6. **Mini App** inside Facebook/Instagram
7. **In-app Analytics Suggestions** (e.g., “this product is trending”)
8. **Multi-Warehouse Stock Support** (premium vendors)

9. Headless Commerce API (GraphQL or REST)

MULTI-TENANT CONTROL & ROUTING

- Each vendor store assigned a **unique store ID** and **subdomain**
 - Super Admin controls store-server allocation with a routing map
 - Custom domain setup flows via Cloudflare or CNAMEs + automated certs
 - Usage metrics and server load determine if a tenant is **migrated to another instance**
 - **Load balancing** via NGINX + Cloudflare Page Rules or Argo Smart Routing
 - CDN edge caching ensures no backend hit for 90% of public storefront traffic
-

Shipping & Courier API

1. Courier Integration for Bangladesh

Integrate major Bangladeshi courier services using their available APIs to automate logistics workflows:

Popular Courier APIs to Integrate:

- **Pathao Courier** (API available)
- **Paperfly** (API available)
- **Redx** (API available)
- **Steadfast Courier**
- **eCourier**
- **Delivery Tiger**
- **Shundorbon Courier** (manual support)

 These APIs can automate:

- Order syncing
- AWB creation
- Shipment tracking
- Status updates
- Pickup requests

How It Will Work:

Vendors can select their preferred courier(s) from their dashboard and configure:

- Pickup address
- Default weight & parcel dimensions

- Auto-generate courier booking on order placement
 - Tracking page integration
 - COD management
-

2. Manual Shipping Method

Not all vendors will want courier integration. Some may:

- Handle deliveries personally
- Work with couriers without APIs
- Only ship locally

Manual Shipping Options to Offer Vendors:

- **Flat rate shipping** (e.g., ₳60 per order, nationwide)
- **Local pickup** (with address and pickup time instructions)
- **Zone-based rates** (e.g., Inside Dhaka vs Outside Dhaka)
- **Weight-based shipping** (e.g., ₳50 for 0–1kg, ₳100 for 1–2kg)
- **Order value-based free shipping** (e.g., free delivery over ₳1000)
- **Cash on Delivery toggle**
- **Custom delivery note for each order**

Vendors can fully manage these from their dashboard — no coding or API needed.

Super Admin Control

- Approve/deny courier connection requests (in case of business verification)
 - Enable/disable manual shipping globally
 - Set default rates and override permissions
 - Monitor courier API usage per vendor
-



Suggested Improvements to Stand Out:

- **Auto-track courier performance per vendor** (delay, delivery success rate)
- **In-store delivery radius mapping** using Google Maps (great for local vendors)
- **Vendor-specific delivery calendar** with holidays/closed days
- **Courier recommendation engine** (based on region, delivery time, and cost)